

Log Aggregation, Analysis and Diagnostics Platform

Team 1: Emmanuel Dairo, Sam Watts, Martin Kelly, Addie Tweed, Ahmed Ikram, Sarah Kelly, David Hyacinth-Ngwu
CS32002 Industrial Team Project
Discipline of Computing
University of Dundee

Abstract - Modern ATM networks generate large volumes of operational data across multiple channels, yet the absence of unified tooling means anomalies frequently go undetected until they cause service disruption. This report presents the design, implementation, and evaluation of a log aggregation, analysis and diagnostics platform developed for NCR Atleos. The system ingests synthetic log data from seven sources, normalises them into two unified schemas (metrics and events), and applies a rule-based detection engine to identify seven defined anomaly types. Detected anomalies are ranked by a weighted criticality algorithm and presented through a React-based dashboard, with each anomaly accompanied by a root cause explanation, operational impact assessment and recommended remediation action. Role-based access control and JWT authentication secure the system. A comprehensive test suite of 48 (passing) tests, spanning unit, integration, concurrency, security and data generation tiers, uncovered and resolved several critical defects prior to release. User evaluation confirmed the interface was intuitive and the recommended action feature particularly valuable. The four core project requirements were met along with some selected stretch goals such as data retention configuration, cross-source anomaly detection and role-based access to name a few.

1 Introduction

There are almost 50,000 ATMs in the UK [1], with the average UK adult withdrawing £1352 from these cash machines in 2025 [2]. If any issue occurs on an ATM, there is immediate impact, with customers losing access to cash and creating a loss of trust in the service. To identify any such problems, ATMs produce data from various sources, such as software logs, operating system metrics, hardware health indicators and event streams. These logs are used to provide meaningful insights for ATM managers to understand any issues that arise in their ATM network. The aim of this project was to create a system that aggregates these logs, provides meaningful analysis on the data, then presents the information clearly. There were four main pillars to this project:

The system must be able to accept logs from multiple sources. Due to the protected nature of real NCR Atleos data, it cannot be used, and instead, synthetic log data must be generated from provided data dictionaries.

From the ingested logs, analysis must be performed. This may include detecting anomalies, correlating events across sources, and identifying indicators of problems. The system must display dashboards and views. These could include charts, timelines and summaries of key events. The system must also clearly highlight detected anomalies.

The system should also be able to provide automated recommendations for possible root causes of anomalies, and suggested actions.

Beyond these four core requirements, the technology choices the project team could use were open, and third-party libraries were allowed, as long as they were free for commercial use and no external servers were used, the entire system should be able to run locally.

The project team were given roughly 3 weeks to develop this system, including design, development, testing and evaluation. The goal of this report is to document the project team's development process for this system, including design decisions, implementation strategies, testing and evaluation, and finally a conclusion.

2 Background

Originally logs from applications used to either be sent to the system console or to private files, before later being standardised to syslog, where messages and related severity were instead registered within 'syslogd'. Some of the first explicit applications for managing logs came into notoriety as the internet was starting to scale rapidly, wherein businesses had to swiftly accommodate their network structures around this substantial increase in logs. Splunk was one of the most popular: made up of what would be three of the four cornerstones of log management: collection, aggregation and retention. It was missing analysis.

Logs need to be available, properly digested and assessed within a variety of contexts for numerous potential reasons specific to a business or entity.

Operational maintenance is one of the main areas, where logs are used to ensure proper functioning of systems within a network. They'd also be used to enable diagnosis for issues within a system under the circumstances something is malfunctioning or not functioning at all.

Security is another pillar to consider. Logs can be used to display indications of an attack that has happened or is

ongoing. It can also expose potential security risks that lie within the system and enable it to be further developed upon. This is tangentially related to the next main category that log assessment is used for: regulatory compliance.

Activity within an organisation that may be a causation for security breaches; defined as a leak, can be identified through logs. Establishments where correct functionality is abs such as health organisations may be required to have regular activity and audit assessments for their systems.

Going by the brief, the anomalies that are targeted are primarily related to operational functionalities; and as such the software is going to be leaning specifically to that specialty given the resource and time limitations. While a regulatory compliance or security flaw could be a cause of operational falter, the focus is still on operations as opposed to, for example, maintaining a secure network. LogRhythm in contrast was another example of a log management application from a similar time to Splunk, instead with a specific focus on security. The system could still be relatively easily expanded past the maximum scope of the project if so was desired to fit into accommodating for other specific areas.

Datadog is arguably one of the most prominent log management applications, designed to facilitate the largest of scale monitoring with a multitude of features, some of which are worth acknowledging, though the biggest acknowledgement is that these are not run locally but instead via their own servers.

Beyond just dealing with log aggregation, it also allows you to configure it with different filtering and normalisations, as well as allowing designating log and stream sources to different aggregation instances

You can configure the types of users and what information they can see. The benefits this provides is mainly two-fold: Firstly, it provides better security, such that the system can be used to present potentially sensitive information that not everyone within an organisation can see. Secondly, it allows the usability for users of the system to be more streamlined and bolsters user experience. They're not being presented information that is of no pertinence to them and the next feature can be individually pertained.

There is a fleet of different graphs that can be used to display statistics relating to logs.

One big focus the system has is allowing the admin to configure and sort how certain pieces of information get visualised, be it trend charts for operations or line graphs for active CPU load.

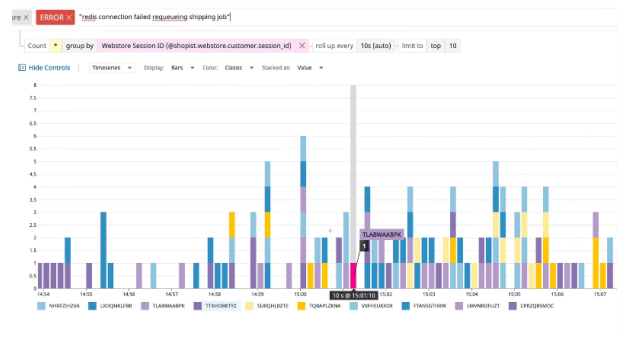


Figure 1: Datadog graph image

One of its main competitors brings attention to another factor yet to be brought up: Pricing

“Better Stack” presents itself explicitly on its websites and promotional material as being “30x Cheaper than Datadog” [3]. No interest in subtlety. Being these are both cloud platforms they come at a monthly cost, per host. This is perfectly suited to larger businesses as it’s perfectly positioned as being rather expensive.

As for locally run solutions, their version of pricing competition comes in the form of storage promises. Open Observe pits itself as being “140x lower storage costs compared to Elasticsearch.” Both locally run log aggregators. Aggregating logs overtime can put costs on a business just through the storage requirements it begs for, and so smaller files is a similarly large consideration.

One social and ethical issue that must be brought up is the increasing providence of AI features and their uses and pitfalls. This of course is far from exclusive from the world of log management, it’s everywhere. However the acknowledgement of the hysterical amount of energy and water consumption it requires contributing to climate change, the further contributing to the cost-of-living crisis by increasing the costs of computer hardware as demanded by the datacentres required for AI to run and job loss is still worth considering. Regardless, there are some considerations specific to log management that is worth considering, and that is the danger of assuming that AI is correct in relation to analysis summaries, explanations, sorting or whatever other feature they’re put into. AI in its current form is still unreliable and is provably false or misguided in many instances and so misuse of the feature could unintentionally work against maintaining ongoing operations, security and regulatory compliance, be it by giving false or misleading flags relating to anomalies or bad advice in relation to analysis.

3 Specification

3.1 Problem Definition

Modern banking infrastructures, particularly those supporting self-service systems generate a huge volume of operational logs. These data streams come from multiple channels, including ATM hardware sensors, their cloud system, event streams, operating system metrics, software application logs. While they have this huge useful source of data which could improve the

systems health, banks currently struggle to turn those logs into actionable intelligence. The core challenge is that banks possess the data, but they lack the correct pipeline to turn those data from multiple sources into something insightful, something that can identify patterns, correlate events across multiple sources/platforms, or visualise anomalies in real-time. This problem impacts operational teams who are forced to manually look through the huge pile of data to identify an anomaly in a system. It also affects customers who experience operation downtime due to engineering teams not quickly spotting an anomaly. Lastly, security unit teams would find it difficult to detect malicious activities going on in the system because the system couldn't spot real time anomalies. This problem matters because unprocessed logs can lead to missed opportunities to predict and prevent an anomaly from occurring. For instance, a hardware health indicator might have signal that there would be an ATM failure days before it occurred; if this signal is missed, the bank would need to pay a high repair cost and may lose revenue from transactions. Furthermore, the ability to distinguish between an anomaly and a cyber-attack would be crucial for protecting the banks reputation. Failing to ingest the data and make a useful insight from it won't just be a technical oversight, it could be a huge risk for the business.

3.2 System Requirements

Based on the background review, the following requirements are deemed important to solve the problem. The full initial planning documents and extended requirements definitions are provided in Appendix E.

Functional requirements:

- FR1: The system shall be able to ingest logs of different formats.
- FR2: The system shall allow data from both log-files based sources as well as event streams.
- FR3: The system shall be able to normalise the different log data into a unified format.
- FR4: The system shall save all normalised log records into a database.
- FR5: The system shall automatically detect anomalies in the ingested log data.
- FR6: The system shall report identified anomalies to the user via a dashboard interface.
- FR7: The system shall display which ATMs are currently non-functional or have warnings.
- FR8: The system shall generate a probable root cause for detected anomalies
- FR9: The system shall identify anomalous patterns in log data across different channels.
- FR10: The system should use identified patterns to predict potential future anomalies.
- FR11: The system should display recommended next actions for anomalies.
- FR12: The system should allow users to filter data and alerts by specific ATM-IDs.
- FR13: The system should provide a configurable data retention period, after which records are purged.

Non-functional requirements:

- NFR1: The system shall enforce role-based access control, distinguishing between standard user and administrator privilege levels.
- NFR2: The system shall handle malformed or incomplete log records without crashing.
- NFR3: The system shall support concurrent ingestion from multiple log sources without data loss or database corruption
- NFR4: The system shall preserve unresolved anomaly records regardless of data retention period.
- NFR5: The system shall provide a usable interface for both technical and non-technical users.
- NFR6: The source code shall be well commented, version-controlled and structured to support future maintenance and development.
- NFR7: The system should be extensible, allowing new log source types to be added without modification to the core log ingestion pipeline.

3.3 Initial Project schedule

Week	Goal
1	Designing the system, writing requirements, creating wireframes, and planning allocation of tasks.
2	Building a functional system, integrating each part and making sure the whole system works seamlessly.
3	Testing the system, conducting evaluation, prepping to present the system to the client, and writing the report.

3.4 Resources

For the backend, Python was used as the primary programming language, FastAPI was chosen as the backend framework for handling API requests and communication with the frontend, and APScheduler was integrated within the backend to manage background tasks. SQLite was used for the database along with WAL to improve read/write performance. For the frontend, React was used to build a responsive UI, and styling was implemented using standard CSS rather than frameworks such as tailwind.

3.5 Deliverables

1. A report which describes the project.
2. The application and source code.
3. A manual on how to use product.

4. Minutes of meetings with management and client.
5. Project presentation material.
6. Evaluation and design materials.

4 Design

To design a system that would address the problem, the first step was to create personas. This aimed to identify who the primary users of the system would be to ensure the key user needs were acknowledged. Originally, five user personas were created (the complete set of five is detailed in Appendix E), but this was narrowed down to the three users who the team believed would benefit most from the system: a manager, a data analyst, and a software engineer.

Persona 1

Name: Steven Smith

Role: Manager

Age: 30

Background: A business management undergraduate who is new to the job and doesn't have a technical history.

Goals:

- Understand system performance through clear dashboards and insightful summaries.
- Make data-driven product and operation decisions based on insights from log analysis.

Pain Point:

- Requires easy to understand summaries (not too technical).

Key Features Needed:

- Clear dashboard and visuals.
- Simple explanation of anomalies detected.

Persona 2

Name: Lionel Torvos

Role: Data Analyst

Age: 35

Background: Experienced data analyst with a master's degree in data science. Comfortable working with large datasets, statistical analysis and data pipelines.

Goal:

- To easily access structured log data for analysis.
- Identify patterns, correlations, and anomalies across multiple log types and sources.
- Support engineering and management teams by providing meaningful insight from the data.

Pain Point:

- Logs from multiple sources are often inconsistent or unstructured.
- Organising data can take a long time before analysis can begin.
- Difficult to correlate events across different systems without shared identifiers.

Key Features Needed:

- Structured and consistent log formats.

Persona 3

Name: John Davis

Role: Software Engineer

Age: 50

Background: Worked as a software engineer for multiple companies and has become accustomed to many different systems and software.

Goals:

- Maintain, improve and polish functioning systems.
- Discover and fix bugs in the system.

Pain Point:

- Spending time finding the source of bugs and errors in the system manually.
- Tracking trends in data that might cause long-term system inefficiencies manually.

Key Features Needed:

- Summary of the source and cause of issues.
- Automated detection of anomalies in logs.

After the creation of these personas and defining wider system requirements, the first wireframes were designed. It aimed at addressing these goals and key features (the full set of Figma UI wireframes can be found in Appendix H). The aim was to design a system that was simple, easy to navigate, and intuitive to understand. To achieve this, two main pages were decided on: a dashboard that displayed a list of anomalies (see Figure 2 below), and a data analysis page to dive deeper into a specific anomaly (see Figure 3 below).

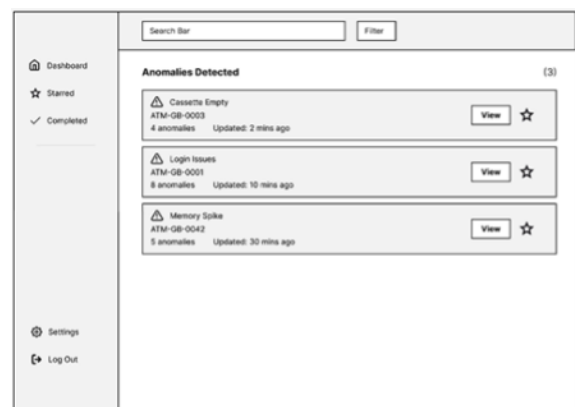


Figure 2: Dashboard wireframe

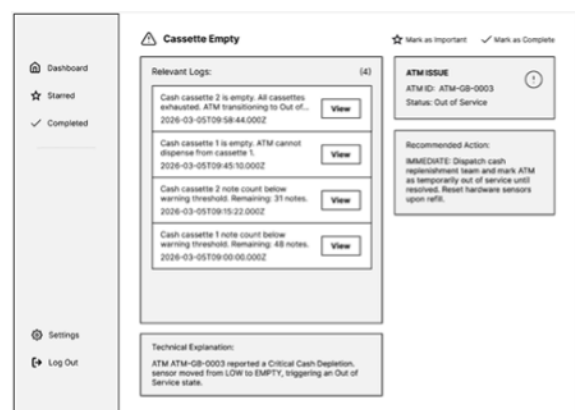


Figure 3: Data analysis wireframe

Extra features were included to benefit user experience, such as the ability to mark an anomaly as "Starred" which provides easy access to anomalies of interest from the side navigation bar, and also the ability to mark an anomaly as "Completed" which removes it from the

dashboard of active anomalies and again into its own, easily accessed section.

Upon meeting with the client, the team received feedback that the design fit their requirements, with a focus on the lack of clutter and easy to understand interface, and advice not to concentrate solely on ATMs as issues could occur in other systems, such as servers, too.

As none of the members of the team had great knowledge of frontend UI development, research was launched into frameworks to build a tech stack that didn't have a steep learning curve. This resulted in the choice of React, which was equipped with beginner manuals, tutorials, and a large online community [4][5]. Pure CSS was opted for styling instead of dedicating extra time learning to navigate a CSS library like Tailwind, which might have greater efficiency longer term.

The backend was written in Python, a language the team agreed everyone was familiar with and connected to the frontend by FastAPI. FastAPI was chosen, instead of another framework like Flask, because of features like automatic document creation and built in APScheduler used for background tasks like data retention. For the database, SQLite was chosen as it was well known by the team and simple to set up. It also required no external interface. However, SQLite prioritises deployment over high-concurrency handling, which required configuring WAL and the implementation of a custom retry with backoff mechanism to allow write functions to handle locks without data loss.

User roles were designed for the system, allowing some users to have admin privileges. An admin user has access to an extra page on the side navigation bar that allows them to set the data retention period of the database and to create new users, including other admins.

With the tech stack decided and wireframes finalised, the team began the second sprint. The wireframes were hardcoded in JavaScript, providing a basic, functional visualisation of the system's UI (see Figure 4 below). Early on, the backend was planned to handle the logging and analysis of all the data, and the frontend would only handle display, with simple formatting of data such as the time received for better readability.

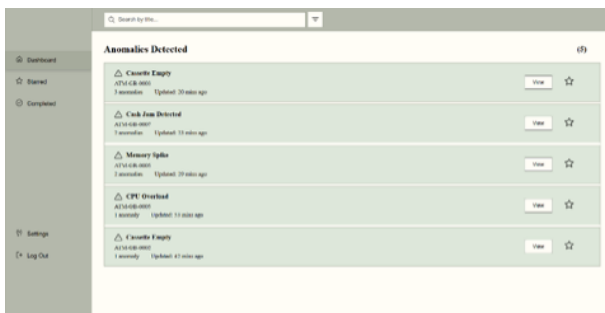


Figure 4: First version of UI

From this initial implementation, team members working on other parts of the system were able to give feedback

based on their own progress. This led to the removal of viewable logs from the data analysis page as correlated logs were already being grouped into one anomaly at the detection stage and therefore reached the frontend as a singular event. Returning to the user personas and requirements, the data analysis page was redesigned to display the most relevant data to the system's users: a root cause, impact to operation, a recommended fix, and anomaly details (such as the source, severity, and time received). Also, instead of anomalies appearing in a random order on the dashboard, an algorithm was created to display these based on how critical an anomaly is. This algorithm took severity, anomaly type, and time received into account when ordering the anomalies.

In the third sprint, the UI was designed to have a more appealing and professional display, with a minimalist, sleek colour palette of blue and white. The final step of the design was to get user feedback through carrying out an evaluation (see Section 7: Evaluation). The team received usability feedback such as that the system could benefit from implementing a back button rather than defaulting on the browser's navigation, there was no way to remove an anomaly from the "Completed" state, and that the filtering system was unclear on the dashboard. This feedback was taken into consideration, the benefits of implementing these new features weighed, and then implemented where applicable (such as was the case with the above-mentioned feedback).

From this user testing, the team was able to confidently settle on the final design that met the user needs. The resulting dashboard (see Figure 5 below) provides an intuitive experience with a minimalist UI and clear descriptions of each page. This carries over to the final data analysis page which has a clearly laid out anomaly breakdown and recommended action, presenting the user with a solution easy to identify on the page (see Figure 6 below).

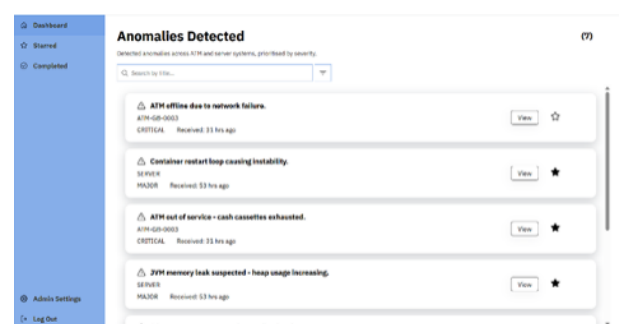


Figure 5: Final dashboard UI

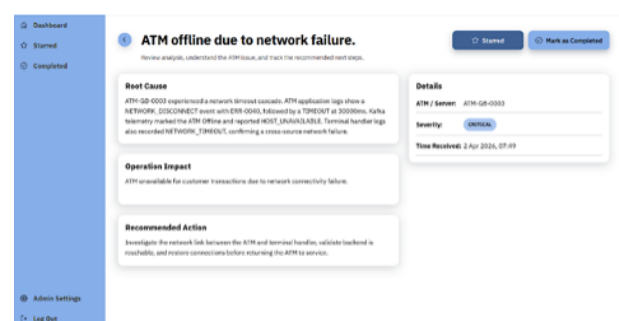


Figure 6: Final data analysis UI

5 Implementation

Implementation responsibilities were distributed to leverage the team's strengths. Frontend UI development and React Integration were primarily handled by Sarah supported by Sam and Ahmed. Backend development was divided systematically: Martin focused on the anomaly detection logic, Emmanuel and Addie developed the ranking algorithms and the analysis router, while Ahmed implemented the remaining backend APIs, the database and the log ingestion pipeline including the synthetic log generator.

The development process followed an agile workflow, guided by defined user stories and epics (provided in Appendix E) with short iterations and continuous feedback. The team conducted daily stand-ups to communicate progress, identify blockers, and align on priorities. Before client meetings, the team held internal discussions to prepare questions and ensure clarity in requirements. The implementation was structured across three main phases. A complete record of minutes from all client and management meetings are provided in Appendix G.

During week one, the initial system design was developed, including wireframes for the dashboard and database schema preparation. Client feedback was incorporated early to validate design decisions.

During week 2, the full system was implemented, including backend anomaly detection, log ingestion, anomaly APIs, analysis APIs, and frontend dashboard. The focus was on building a functional end-to-end system for early client review.

During week 3, final adjustments were made based on the client feedback, alongside system testing and evaluation. The team also worked on the report and prepared the presentation.

Throughout the process, priorities were adjusted, especially during week 2 where delivering a complete working system was the top priority.

5.1 System implementation overview

The system was implemented as a full-stack application consisting of a Backend, responsible for anomaly detection, analysis, data processing, and API endpoints; a database, where SQLite was used for storing anomalies and processed data; and a frontend, providing a dashboard interface for visualising anomalies and system insights. The backend was designed in a modular structure, separating detection logic, analysis and ranking, and API routing. This separation improved maintainability and allowed independent development of components.

5.2 Key implementation features

A rule-based anomaly detection system was implemented to identify predefined scenarios (A1 - A7). Each anomaly type was encapsulated in its own function, improving readability and modularity.

The detector processes unified log data and applies rules such as event correlation across multiple sources (e.g., ATM logs, Kafka, system metrics [6]), threshold-based checks (e.g., response time, memory usage), and sequential event validation. This approach allowed complex real-world behaviours (e.g., cascading failures) to be modelled in a structured way.

5.2.1 Ranking algorithm

One of the significant components implemented was the anomaly ranking algorithm, designed to prioritise critical issues.

Instead of treating anomalies equally, the team chose to use a weighted scoring system to assign priority:

```
37 # implement the operation affected gravity mapping
38 operation_ranking = {'A1': 7, 'A4': 6, 'A3': 5, 'A2': 4, 'A6': 3, 'A5': 2, 'A7': 1}
39
40 # severity breakdown mapping
41 severity_map = {"CRITICAL": 3, "HIGH": 2, "MAJOR": 1, "WARNING": 0}
42
43 # transaction in progress mapping
44 transaction_map = {'is_transaction_t': 1, 'is_transaction_f': 0}
45
46 combined_weights = {}
47 reference_time = get_reference_now(anomalies)
48
49 # to improve performance, i would create kv pair of anomaly so i can get a better search
50 # anomaly_map = {}
51 counter_id = 0
52 atm_tracker = {}
53
54 # combine weights to get the total weight for ranking
55 for a in anomalies:
56     counter_id += 1
57
58     #time map
59     time_score = get_age_score(a.get('detected_at'), reference_time)
60
61     # get score without frequency to rank anomalies in atm
62     inner_score = (
63         operation_ranking.get(a['anomaly_type'], 0) +
64         severity_map.get(a['severity'], 0) +
65         (transaction_map.get('is_transaction_t', 0) if a['transaction_id']
66         time_score
67     )
68
69     #just did a little testing with the weight, the weight currently makes a little dif
70     # print(inner_score - time_score)
71
```

Figure 7: Weighting snippet

Key design decisions included operational impact weighting, where certain anomaly types (e.g., A1) were prioritised higher; severity mapping, such as CRITICAL issues ranked above WARNING-level issues; time mapping, where old anomalies were prioritised using a time-based scoring function; and transaction awareness, where active transaction issues were given higher importance. This produced a ranked list of anomalies, enabling the frontend to display the most critical issues first. A notable challenge was balancing the weights assigned to ensure no single factor dominated the ranking unfairly (e.g., time mapping using conditional weighting rather than time weight). The final ordering was achieved by iterative testing and adjustments.

5.2.2 Detailed analysis generation

To make anomalies interpretable, the analysis system generates detailed analysis for each anomaly type,

including an explanation of probable root cause, its operational impact, and recommended action. For example, A1 (network failure) produces a structured explanation combining multiple signals:

```
# the anomaly explanation in details
explanation = (
    f"Container restart telemetry showed repeated restart activity"
    f"{' up to {max_restart:.0f} restart' if max_restart is not None else ''}. "
    f"Handler logs recorded {startup_count} STARTUP event"
    f"{' and {fatal_count} fatal failure' if fatal_count else ''}, "
    "indicating the service repeatedly crashed and restarted within a short window."
)
```

Figure 8: Detailed root cause code

This transformed raw anomaly data into a meaningful insight for end users, making the system more practical for real-world usage.

5.3 API integration

The backend exposes a set of REST API endpoints built with FastAPI across four routers: analysis, anomalies, admin and authentication.

The analysis router provides a single `/detailed` endpoint which returns the full ranked anomalies with detailed explanations attached, serving as the primary data source for the frontend dashboard.

The anomalies router handles the core anomaly management functionality. The `/anomalies` returns a paginated, filterable anomaly list (supports grouping). Individual anomalies can be resolved via the `/anomalies/{id}/resolve` endpoint which marks anomalies resolved or unresolved. The `/anomalies/{id}/star` endpoint provides a toggle to star or unstar an anomaly for later reference.

The admin router manages system configuration and user management. The `/retention` endpoint supports GET and PUT methods. A `/cleanup/wipe` endpoint is provided to permanently deletes all rows from the database tables and runs a VACUUM operation to reclaim storage. New persistent users, including other administrators, can be created through the `/users` endpoint.

The authentication router handles user access. The `/auth/login` endpoint validates credentials and grants access to authorised users, while `/auth/me` returns current user's information. New user accounts can be registered via `/auth/register`.

5.4 Challenges and problem solving

One key challenge was handling incomplete or inconsistent data. In several cases, expected fields were missing from logs, requiring fallback logic and defensive programming. Another challenge was correlating events across multiple data sources. For example, detecting a network failure required confirming signals from ATM logs, Kafka streams, and Terminal handler logs. Ensuring these signals were aligned correctly required careful creation of detection logic. Additionally, performance considerations led to optimisations such as using

dictionary mappings for faster lookups and avoiding repeated scans of anomaly data.

5.5 Reflection on Implementation

The implementation evolved significantly from the initial plan. While early designs focused on completeness, later stages prioritised delivering a functional and interpretable system. Client feedback influenced several improvements, especially concerning dashboard structure, clarity of anomaly explanations, and prioritisations of critical issues. Overall, the final system successfully integrates detection, analysis, and presentation.

The complete version-controlled source code for the system is provided in Appendix A.

6 Testing

Testing responsibilities were distributed across the team. The anomaly detection tests (test_anomaly_detector.py) were implemented by Martin, the analysis endpoint tests (test_analysis_endpoints.py) by Addison, and all other tests were developed by Ahmed.

6.1 Testing Strategy

NCR Atleos did not prescribe a formal testing methodology, so the team defined a strategy aligned with the system's primary risk areas: continuous data ingestion under concurrent load, correct anomaly detection without excessive false positives and graceful handling of malformed data. The strategy was structured across five tiers: unit, integration, concurrency and stress, security and data generation validation - all implemented using the pytest framework [7]. Tests were written incrementally alongside development rather than retroactively ensuring errors were caught at the point of introduction. The test suite comprises 48 passing tests (see Figure 9 below.)

```
(.venv) ahmedikram@MacBook-Pro log-aggregation-analysis-and-diagnostics-platform % pytest
===== test session starts =====
platform darwin -- Python 3.14.3, pytest-9.0.2, pluggy-1.6.0
rootdir: /Users/ahmedikram/log-aggregation-analysis-and-diagnostics-platform
plugins: anyio-4.13.0
collected 48 items

backend/tests/stress/test_write_helper_locking_collision.py . [ 2%]
backend/tests/test_admin_retention_endpoints.py .. [ 6%]
backend/tests/test_analysis_endpoints.py . [ 8%]
backend/tests/test_anomalies_endpoints.py ... [ 14%]
backend/tests/test_anomaly_detector.py ..... [ 33%]
backend/tests/test_atm_app_parser.py . [ 35%]
backend/tests/test_auth_endpoints.py ..... [ 50%]
backend/tests/test_auth_security.py .. [ 54%]
backend/tests/test_base_parser.py .. [ 58%]
backend/tests/test_cleanup.py ... [ 64%]
backend/tests/test_concurrent_writes.py . [ 66%]
backend/tests/test_db_connection_pragmas.py . [ 68%]
backend/tests/test_db_schema.py ..... [ 79%]
backend/tests/test_generator_scenarios.py . [ 83%]
backend/tests/test_hardware_sensor_parser.py . [ 85%]
backend/tests/test_ingestion_integration.py . [ 87%]
backend/tests/test_kafka_metrics_parser.py . [ 89%]
backend/tests/test_prometheus_parser.py . [ 91%]
backend/tests/test_terminal_handler_parser.py . [ 93%]
backend/tests/test_windows_os_parser.py . [ 95%]
backend/tests/test_write_helper_retry.py .. [100%]

===== 48 passed in 10.44s =====
```

Figure 9: `pytest` terminal output

6.2 Unit Testing

Unit testing formed the bulk of the testing process, isolating individual components to verify correctness independently of the wider system.

Parser Correctness: A dedicated test module was written for all seven ingestion sources (`test\_atm\_app\_parser.py`, `test\_hardware\_sensor\_parser`

.py', 'test_kafka_parser.py', 'test_prometheus_parser.py', 'test_windows_os_parser.py', 'test_gcp_cloud_metrics_parser.py', 'test_terminal_handler.py') These tests verify that each parser correctly maps raw source fields to the 'events' and 'metrics' tables, normalises log levels to uppercase and converts timestamps to UTC ISO 8601. Notably, 'test_base_parser.py' validates the dead-letter queue (DLQ) behaviour [8], when a malformed record is encountered the ingestion pipeline must route it to the 'ingestion_errors' table and continue processing rather than raise an unhandled exception / crash.

Database Integrity: 'test_db_schema.py' verifies that all tables, indexes, primary keys and foreign key constraints are present and correctly enforced after initialisation. 'test_db_connection_pragmas.py' validates that every database connection opened with WAL journal mode [9], a 'busy_timeout' and foreign key enforcement - pragmas that are critical for safe concurrent access and referential integrity.

Utilities: 'test_write_helper_retry.py' tests the resilience of the batch write mechanism against short-lived SQLite lock errors and 'test_cleanup.py' confirms that the data retention routine correctly purges records beyond the configured age threshold without affecting newer records.

6.3 Integration Testing

Integration tests verified the communication between the ingestion pipeline, SQLite database and FastAPI backend under realistic request-response conditions. The FastAPI 'TestClient' was used to exercise API endpoints against an in-memory test database, allowing full endpoint logic to be tested without a running server.

'test_ingestion_integration.py' validates the complete ingestion pipeline: a raw synthetic payload is routed through the correct parser, normalised and committed to the database, after which a query confirms the record is retrievable with the expected field values. Endpoint-level integration tests cover the anomalies ('test_anomalies_endpoints.py'), analysis and timeline (test_analysis_endpoints.py), and admin retention configuration (test_admin_retention_endpoints.py) routers. During this phase, a critical field mismatch was discovered, the backend was emitting 'anomaly_code' while the frontend expected 'anomaly_type', this was identified and resolved - through integration testing - before it could affect the final system.

6.4 Concurrency and Stress Testing

SQLite's file level locking model poses a real concurrency risk for a system ingesting from multiple sources simultaneously. Two dedicated test modules address this. 'test_concurrent_writes.py' simulates multi-threaded writes to confirm that simultaneous ingestion does not cause data loss or 'database is locked' errors under normal load. The stress test module 'stress/test_write_helper_locking_collision.py' goes further, deliberately creating lock collisions under heavy

load to verify that the retry-with-backoff mechanism in 'write_helper.py' recovers correctly (see Figure 10.)

```
def write_batch(conn: sqlite3.Connection, sql: str, rows: Iterable[tuple],
               retries: int = 5, backoff_base: float = 0.1, backoff_max: float = 2.0) -> None:
    """Write a batch of rows using provided connection with retry/backoff.

    - 'conn' must be an sqlite3.Connection (obtained from database.connection.get_db()).
    - 'sql' is the parameterised insert SQL.
    - 'rows' is an iterable of parameter tuples.
    """
    attempt = 0
    rows = list(rows)
    while True:
        try:
            cur = conn.cursor()
            cur.execute('BEGIN')
            cur.executemany(sql, rows)
            conn.commit()
            return
        except sqlite3.OperationalError as e:
            if 'locked' in str(e).lower() and attempt < retries:
                # Ensure any open transaction is rolled back before retrying
                try:
                    conn.rollback()
                except Exception:
                    pass
                wait = min(backoff_max, backoff_base * (2 ** attempt))
                logger.warning('Database locked during write; retrying in %.3fs (attempt %d)', wait, attempt + 1)
                time.sleep(wait)
                attempt += 1
                continue
            logger.exception('Write batch failed: %s', e)
            try:
                conn.rollback()
            except Exception:
                pass
            raise
        except Exception:
            logger.exception('Unexpected error during write_batch')
            try:
                conn.rollback()
            except Exception:
                pass
            raise
```

Figure 10: 'write_batch' function from 'write_helper.py'

This function ensures the pipeline recovers gracefully rather than failing silently when multiple ingestion threads compete for the same write lock.

6.5 Security and Authentication Testing

'test_auth_endpoints.py' covers the core authentication flows: login, JWT token generation and the '/auth/me' endpoint. 'test_auth_security.py' tests security boundaries more rigorously including invalid JWT claims, privilege escalation attempts (a regular user accessing admin-only routes), and malformed payload handling. These tests verify that the RBAC guards ('require_admin' and 'get_current_user') behave correctly at the dependency injection layer.

6.6 Data Generation Validation

'test_generator_scenarios.py' validates the synthetic data generator (custom_data_generator.py) to ensure it produces statistically consistent output with correctly injected anomaly scenarios (A1-A7), realistic cross-source correlation via 'correlation_id' and properly threaded 'transaction_id' values. This test module is particularly important for system-level confidence, as the entire ingestion pipeline is exercised against generator output, so inaccuracies in the generator would propagate into other test results.

6.7 Issues Uncovered During Testing

The automated test suite gave huge value in being able to reveal defects in the system that may otherwise have gone undetected, some examples include:

Silent Data Loss Under Concurrent Load ('stress/test_write_helper_locking_collision.py'): SQLite only allows one writer at a time, and concurrent writes can cause locking issues [10]. When the Kafka, Prometheus, and OS metrics parsers run concurrently and

attempt simultaneous inserts, a missing retry mechanism would cause writes to fail with `'sqlite3.OperationalError: database is locked'` and drop records silently. The locking collision stress test exposed this by creating fifty concurrent write threads simultaneously, proving that the exponential backoff logic in `'write_helper.py'` was necessary to guarantee all records eventually reach the database. Without it, the system would operate on an incomplete log dataset.

Deletion of Unresolved Anomalies ('test_cleanup.py'):

A naïve implementation of the data retention cleanup would run a blanket delete against the `'anomalies'` table based solely on age. Testing revealed that this would silently remove active, unresolved anomalies - for example, a critical hardware failure logged thirty-two days ago that a technician had not yet addressed would disappear from the dashboard entirely. The cleanup logic was corrected to preserve anomalies where `'is_completed'` is false, regardless of age, so that unresolved alerts are never purged.

Privilege Escalation via Valid but Unprivileged Tokens ('test_auth_security.py', 'test_admin_retention_endpoints.py'): The admin retention endpoint (`'PUT/admin/retention'`) was initially only verifying that a request carried a valid JWT, without checking the role claim within it. A standard user with a valid token could therefore invoke admin operations including reducing the data retention window to purge the database prematurely. Tests logging in as a non-admin and issuing the `'PUT'` request confirmed the endpoint was incorrectly returning `'200 OK.'` The `'require_admin'` RBAC guard was added to the route after which the same request correctly returned `'403 Forbidden.'`

Brittle Parser Crashes on Schema Drift ('test_kafka_metrics_parser.py', 'test_prometheus_parser.py'):

Parser tests feeding deliberately malformed payloads (missing fields, unknown keys, strings in place of numeric values and empty objects) revealed that several parsers were using strict dictionary access (`'payload['field']'`) rather than safe `'.get()'` calls. A missing field in an incoming Kafka stream would raise a `'KeyError'` and crash the entire ingestion process for that source. The parsers were all updated to use `'.get()'` with appropriate defaults, isolating bad records to the `'ingestion_errors'` table rather than halting ingestion.

Anomaly Field Name Mismatch ('test_anomalies_endpoints.py'): Integration tests against the anomalies endpoints revealed that the backend was emitting the `'anomaly_code'` field in its response while the frontend expected `'anomaly_type'`. This logic error would have silently broken anomaly rendering on the dashboard but was caught and corrected before affecting the final system.

6.8 Limitations and Areas of Improvement

The test suite is comprehensive at the backend but has two notable gaps. First, the React frontend currently contains

no automated tests, using Vitest for component-level unit tests and Cypress or Jest for end-to-end user flow testing [11] would provide coverage of the presentation layer. Second, the integration tests run entirely within the backend via the `'TestClient'`, meaning no true cross-network end-to-end test exists that exercises the frontend communicating with a live API server. These gaps represent a reasonable trade-off given the project timeline but would be one of the first areas to address in a follow-on phase.

The full source code including all automated testing is provided in Appendix A

7 Evaluation

7.1 Evaluation Methodology

The system was evaluated using three participants: two undergraduate computer science students and one undergraduate business student. This allowed the team to get both technical and non-technical feedback. The evaluation was done in person, where participants were asked to use the system to complete basic tasks such as logging in, navigating the system, and viewing anomaly details.

The team used a set of structured questions to guide the session, focusing on usability, clarity, and how helpful the system was for making decisions. No personal data was collected, and all response was kept anonymous. This method helped the team understand how real users interact with the system and whether it meets its intended purpose.

7.2 User-centered Design and Iteration

User feedback played an important role throughout the project. During early client meetings, the team prepared questions to better understand what kind of system and interface they expected. Since the team had limited UI/UX experience, this helped guide their design decisions.

One key decision was choosing a simple, wireframe-style interface instead of a dashboard with charts. This was based on client feedback, which suggested they preferred a clearer and more direct layout.

From the final user evaluation, the team made several improvements, including adding back buttons, making filtering easier to find, and improving how anomalies are ranked. Some suggestions, like adding password rules and extra account features, were not implemented due to time limits and focus on core functionalities.

7.3 Evaluation Results

Overall, the feedback from participants was positive. All users found the system easy to use and said the navigation was simple and clear. The login process was also straightforward, although some users mentioned that security could be improved with features like multi-factor authentication. The dashboard was generally easy to understand, and users were able to identify anomalies without difficulties. The most useful feature mentioned

was the recommended action, as it helped users know what to do next.

However, one issue that stood out was the filtering feature. None of the participants noticed it easily, which shows it was not clearly designed or placed.

The full list evaluation questions, participant feedback and changes made due to evaluation feedback can be found in Appendix D.

7.4 Discussion of results

The results show that the system works well in terms of usability and basic functionalities. Users were able to complete tasks without confusion, which suggests that the simple interface design was effective. One important insight was how valuable the recommended action feature was. Users relied on it to make decisions, which supports the team's choice of using rule-based system. On the other hand, the issue with filtering shows that not all features were as clear as intended. Also, mixed feedback on security suggests that while the system is easy to use, it could be improved to build more user trust.

7.5 Ethical considerations

The evaluation was carried out in an ethical way. All participants took part voluntarily and were informed about the purpose of the session. No personal or sensitive data was collected. There were no recordings made during the evaluation, and all responses were anonymised. This ensured that participants were comfortable and their privacy was protected.

7.6 Limitations

There are some limitations to this evaluation. First, only three participants were involved, which is a small sample size. This means the result may not fully represent a wider group of users. Finally, the participants were not professionals in ATM operations, so their feedback may not fully reflect real industry needs.

7.7 Additional evaluation

From a technical point of view, the system was designed to be simple and efficient. The rule-based detection system allowed fast responses without needing complex machine learning models.

SQLite was used with WAL mode and a retry system to handle multiple operations. This worked well for smaller usage but may not scale well for larger systems with many users.

FastAPI helped improve performance and made it easier to manage the backend. Overall, the system performs well for its intended scope but would need changes to handle larger-scale use.

8 Description of The Final Product

8.1 System overview

The final system is a web-based application designed to monitor and analyse system logs from ATM networks. Its main purpose is to detect anomalies within logs and present them in a clear and structured way so that users can quickly understand issues and take action. The system focuses on simplicity and usability, showing only the most important information rather than overwhelming users with raw log data. Instead of requiring users to analyse logs, the system highlights problems, explains their root cause, gives operational impact and recommended action. It is mainly intended for roles such as system administrators, engineers, and operational staff who need to monitor the systems health and respond to issues efficiently.

8.2 User roles and access

The system was designed with different types of users in mind, based on the defined personas. These include system administrators, engineers, analyst, and management-level users.

Each role interacts with the system slightly differently. For example, technical users such as engineers and analysts rely more on detailed anomaly information, including root causes and system-level insights. On the other hand, less technical users such as management, would benefit more from less technical insight like the operational impact rather than heavy technical details. The system supports role-based access, meaning different users can be given different levels of access to data. This ensures that users only see information that is relevant to their role, improving both usability and security.

8.3 System interface

The interface was designed to be simple and easy to navigate, with a focus on clarity rather than a complex visualisation.

8.3.1 Dashboard

The main dashboard provides an overview of detected anomalies. Information is presented in a list format where each entry shows key details such as severity level, affected systems, and a short summary of the issue. Anomalies are ranked so that the most critical issues appear first, helping users prioritise their actions.

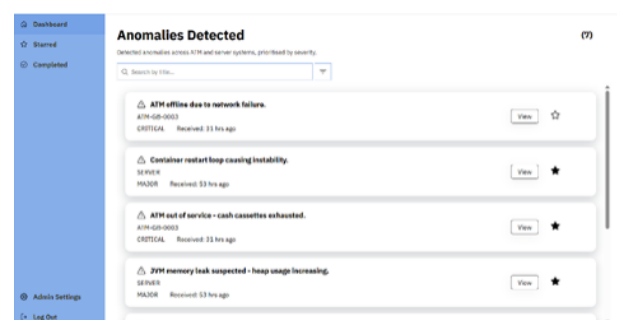


Figure 11: Dashboard Overview

8.3.2 Detailed View

When a user selects an anomaly, they are taken to a detailed view. This page provides more in-depth information, including the root cause of the issue, its potential operational impact, and a recommended action. This feature is important because it removes the need for users to manually interpret logs, instead giving them clear guidance on what to do next.

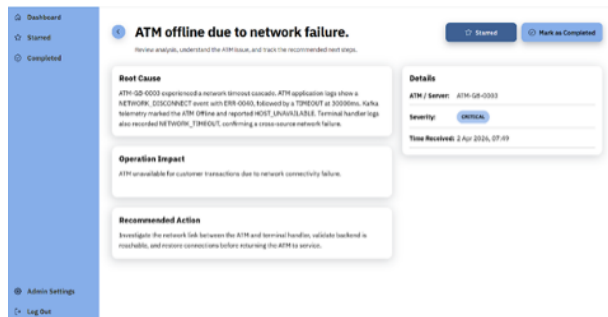


Figure 12: Detailed View

8.3.3 Navigation

The system includes a simple navigation structure, typically using a sidebar. Users can move between different sections such as the main dashboard, completed anomalies, and starred anomalies. Navigation was designed to be straightforward so that users do not feel lost while using the system.

8.3.4 Admin Settings

The system includes an admin only settings screen. Admins can create new users and configure the data retention period from 1 day to 365 days. This is the only place where a new admin user can be created.

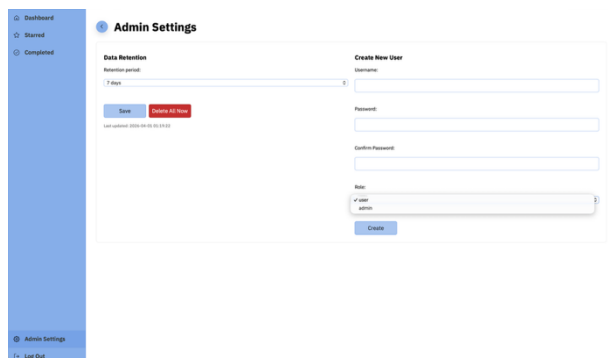


Figure 13: Admin Settings

8.4 Core Features and Functionality

The system includes several key features that support its main goal of anomaly detection and analysis. There is the log ingestion process, where the system collects log data from multiple sources and converts them into a consistent format for easier processing. This is followed by anomaly detection, where a rule-based engine is used to detect unusual patterns or behaviours within the log. After detecting anomalies, they are analysed to provide a root cause, operational impact, and recommended action. The anomalies are then ranked - issues are

prioritised based on factors such as severity, type, and time, allowing users to focus on the most important problems first. Users can filter anomalies to focus on specific systems. Finally, users can mark anomalies as complete or star them for later review.

8.5 User Flow

The system follows a simple and logical flow from the user's perspective: Firstly, the user logs into the system. Then, the dashboard displays a list of detected anomalies. After that, anomalies are ranked so the most critical ones are shown first. The user then selects an anomaly to view more details, and the system provides the root cause, impact, recommended action. The user can then act, and mark the anomaly as completed or save it for later by starring it. This workflow reduces the need for manual log analysis and helps users to make faster decisions. A complete User Manual, detailing system navigation and step-by-step instructions for all features, is provided in Appendix C.

8.6 Backend overview

The backend handles log processing, anomaly detection, analysis, and data storage. Logs from different sources are first collected and converted into a standard format. This allows the system to process data consistently, even when logs come from different systems. A rule-based detection engine then checks the data for known anomaly patterns. When an issue is detected, it is stored along with relevant details such as severity and supporting evidence. The system also includes mechanisms to handle database operation efficiently, ensuring that data can be processed without errors even when multiple operations occur at the same time. Comprehensive instructions for the backend configuration, deployment, and database setup are provided in the Technical Manual (*Appendix B*).

8.7 Design decisions

Several design decisions were made based on project constraints and client feedback.

The first key decision was to prioritise backend functionality over complex visual design. Instead of using charts and advanced dashboards, the system uses a simpler layout that focuses on clarity and usability. Secondly, it was an important choice to implement a rule-based detection system instead of machine learning. This approach was chosen because it is faster, easier to implement, and produces more predictable results.

9 Summary and Conclusions

The project team successfully built a working log aggregation, analysis and diagnostics platform within three weeks, meeting all four core requirements. The system which the team created ingests synthetic logs from multiple sources, detects the seven defined anomaly types, ranks them by criticality, using a weighted scoring system, and finally presents these

anomalies, along with recommended actions, through a clear user interface.

The team felt that throughout the project, their communication was failing them, and if the team were to do the project again, they would better utilise agile techniques to improve their understanding of each other's roles in the system, as well as having a clearer, stricter allocation of tasks. One solution that could be used to improve team communication would be to use a Kanban board to keep track of tasks. This would help by providing a visual overview of what needs to be done during each sprint of development, as well as what has been achieved by the team so far. The team would also plan their sprints more, understanding exactly what needs to be done at the beginning of each week. This would help as the team felt that sprints two and three blended into one. This caused problems because without any clear focus, it was difficult to understand what features needed to be implemented and prioritised. One of the aspects of this project that the team was most proud of was the depth of the system testing, with 48 passing tests across the system, finding real issues that could be solved quickly, without creating technical debt. The project team is also proud of the security of the system, using JWT authentication, and implementing role-based access to allow system admins to have extra control over the system functionality. Another aspect of the development process that the team is proud of is the effectiveness in response to client and user feedback, implementing real changes when a feature wasn't understood or used correctly.

Beyond what was achieved, the project team had some ideas for what they could implement in the system, given more time. Firstly, the team would look to implement predictive detection, using a lightweight ML model, predicting when an anomaly might occur before it does. Another feature the project team would implement would be to allow real-time log ingestion, detecting anomalies as they occur. One more feature the team would implement would be to make the system less ATM-specific, allowing any logging to be processed by the system automatically.

Overall, the project gave the team valuable experience working in an industrial-style agile team, responding to real client requirements under time pressure, which are skills that are crucial for professional software development.

The presentation slides are provided in Appendix F.

Acknowledgments

The project team would like to thank NCR Atleos for providing the project brief, and for their engagement throughout the weekly client meetings and final presentation. The team would also like to thank Brian Pluss for his guidance and support throughout the project. Finally, the team would like to thank the participants who gave their time to take part in the system evaluation.

References

- [1] House of Commons Library, "Cash and digital payments" UK Parliament Research Briefings, [Online] <https://commonslibrary.parliament.uk/research-briefings/cbp-8570/>
- [2] LINK, "Average UK adult withdrew £1,352 from ATMs in 2025" *LINK News*, 2026 [Online] <https://www.link.co.uk/news/average-uk-adult-withdrew-1-352-from-atms-in-2025>
- [3] BetterStack, "BetterStack log manager", [Online] <https://betterstack.com/log-management>
- [4] React Navigation, "Getting Started", *React Navigation Documentation*, [Online] <https://reactnavigation.org/docs/getting-started/>
- [5] React, "Adding Interactivity", *React Documentation*, [Online] <https://react.dev/learn/adding-interactivity>
- [6] Prometheus, "Metric Types", Prometheus Documentation [Online] https://prometheus.io/docs/concepts/metric_types/
- [7] PyPI, *pytest Python Package Index* [online] <https://pypi.org/project/pytest/>
- [8] BrowserStack, "Top React Testing Libraries in 2026," *BrowserStack* Dec. 7, 2025 [Online] <https://www.browserstack.com/guide/top-react-testing-libraries>
- [9] N. Dhandala, "How to Handle Dead Letter Queues in Python," *OneUptime Blog*, Jan. 24, 2026 [Online] <https://oneuptime.com/blog/post/2026-01-24-dead-letter-queues-python/view>
- [10] SQLite, "Write-Ahead Logging," *SQLite Documentation* Mar. 13. 2026 [Online] <https://sqlite.org/wal.html>
- [11] Stack Overflow, "Using SQLite with WAL," *Stack Overflow*, Dec. 2017 [Online] <https://stackoverflow.com/questions/47250220/using-sqlite-with-wal>

Appendices

Appendix A: Source code

The full version-controlled project source code is available on GitHub at:

<https://github.com/SamWatt5/log-aggregation-analysis-and-diagnostics-platform>

or in the Appendix A (Source Code) folder in supporting documentation.

The repository contains the backend and frontend implementations including the anomaly detection and analysis. It also contains other items such as the technical documentation, database and all automated testing.

Appendix B: Technical Manual: Setup and Execution

The system consists of a backend and frontend dashboard UI. The backend is responsible for ingesting log data, normalising and storing the data then detecting anomalies and generating detailed analysis on those anomalies. The

frontend provides a user interface for interacting with the system, including viewing anomalies, filtering anomalies and accessing detailed insights.

Full technical instruction details are included in the Appendix B (Technical manual) folder in supporting documentation.

Appendix C: User Manual: System Operation Guide

Users interact with the system through a web dashboard. Once logged in, users can view a summary of detected anomalies, ranked by criticality. Selecting an anomaly provides access to detailed analysis, like the root cause, operational impact, and recommended action.

Additional workflow details are provided in the Appendix C (User manual) folder in supporting documentation.

Appendix D: Evaluation

The evaluation of the system was conducted to get feedback on the features implemented. Participant provided feedback on usability, clarity, and functionality. A full documentation of the response and implementation based on feedback is included in the Appendix D (Evaluation) folder in supporting documentation.

Appendix E: Requirements

The initial planning and requirement definitions used to guide the development of the system are included in the Appendix E (Requirements) folder in supporting documentation.

Appendix F: Project Presentation

The project presentation slides used to present the system development process and features are included in the Appendix F (Project presentation) folder in supporting documentation.

Appendix G: Meeting Minutes

The complete and comprehensive log of all weekly NCR Atleos client and management meetings, containing the teams client attendance, feedback and discussions are included in the Appendix G (Meeting Minutes) folder in supporting documentation.

Appendix H: UI Figma Wireframes

The complete set of Figma wireframes for the frontend system can be found in the Appendix H (UI Figma Wireframes) folder in supporting documentation.

